Smart Parking Web API (.NET 10 & SQL Server)

Employee Onboarding & Technical Architecture Guide | Solution Path:
C:\Projects\SmartParking\SmartParking.slnx

1. Executive Summary & Purpose

Welcome to the Smart Parking engineering team! This onboarding guide provides a complete walkthrough of the **Smart Parking Web API** built on **.NET 10 (net10.0, C# 14)** and **Microsoft SQL Server (EF Core 10)**. It includes: (1) a detailed explanation of what the application does and how its 4-layer architecture works, (2) a step-by-step guide to building, running, and testing the application on your local machine, (3) key engineering and troubleshooting notes from our initial build session, and (4) the complete original master prompt used to design and generate the platform.

2. Detailed Explanation of What This Application Does

2.1 Business Domain & Multi-Tenant Hierarchy

- **Clients (dbo.Clients)**: Multi-tenant client organizations/operators that manage parking facilities. Supports soft-delete (IsDeleted).
- **Users (dbo.Users)**: Operators and administrators associated with a ClientId. Passwords are hashed using ASP.NET Core IPasswordHasher<User> (PBKDF2) and masked as '***' in API responses.
- **Sites (dbo.Sites)**: Physical properties (malls, tech parks, campuses) belonging to a ClientId.
- **Buildings (dbo.Buildings)**: Individual towers/structures inside a SiteId, created and synchronized transactionally via the nested getVBuildingss collection.
- **Floors (dbo.Floors)**: Parking levels (e.g., P-1, P-2, B-1) within a BuildingId, tracking IsOperational status, TotalParkingSpots capacity, and a SQL Server ROWVERSION concurrency token.
- **Parkings (dbo.Parkings)**: Vehicle check-in/check-out tickets linked to a FloorId and ParkSpotNo. Tracks CustName, CustMobileNo, VehicleNo, ParkDate, InTime, OutTime (NULL while actively parked), Amount, ExtraCharge, and ParkingNo.

2.2 Clean 4-Project Solution Structure (C:\Projects\SmartParking)

Project	Responsibilities & Key Files
SmartParking.Domain	Zero external dependencies. Domain entities (Client, User, Site, Building, Floor, Parking) with RowVersion concurrency tokens and domain exceptions (EntityNotFoundException, ParkingConflictException, InvalidCredentialsException).
SmartParking.Application	All 12 OpenAPI DTO contracts, joined read projection DTOs (SiteListItemDto, FloorListItemDto, ParkingListItemDto), FluentValidation validators, FeatureFlags, CacheTags, RateLimitPolicies, SmartParkingMetrics, and [LoggerMessage] extensions.
SmartParking.Infrastructure	EF Core 10 SmartParkingDbContext, entity configurations, SmartParkingCompiledQueries (EF.CompileAsyncQuery), DatabaseSeeder, SQL Server DDL script (001_InitialSchema.sql), and DependencyInjection.cs (SQL retry strategy, HybridCache, Redis, Polly v8 resilience).
SmartParking.Api	Minimal API endpoint groups (UserEndpoints, ClientEndpoints, SiteEndpoints, FloorEndpoints, ParkingEndpoints), CorrelationIdMiddleware, ValidationFilter, CacheEvictionService, chained IExceptionHandler classes, Swagger UI (/swagger), Scalar UI (/scalar/v1), Health Checks, and Prometheus (/metrics).

2.3 High-Performance SQL Server Indexing Strategy

- **UX_Parkings_ActiveSpot (Filtered Unique Index)**: Created on dbo.Parkings(FloorId, ParkSpotNo) WHERE OutTime IS NULL AND ParkSpotNo > 0. Guarantees at the database engine level that two vehicles can never simultaneously occupy the same active parking spot on a floor, and accelerates GetAllActiveParking.

- **IX_Parkings_Search (Covering Index)**: Indexes (FloorId, ParkDate DESC, VehicleNo) with INCLUDE columns so GetAdminAllParkings executes fast index-only scans.
- **EF Core 10 Compiled Queries & DbContext Pooling**: Uses AddDbContextPool<SmartParkingDbContext> and pre-compiled LINQ queries in SmartParkingCompiledQueries.cs.

2.4 Enterprise Cross-Cutting Capabilities (10 Production Pillars)

Capability	Implementation in SmartParking
1. Health Checks & Metrics	/health/live (liveness) and /health/ready (SQL Server + EF Core DbContext readiness) return structured JSON. SmartParkingMetrics records check-in count, check-out count, active parked vehicles, parking duration histogram, and revenue collected.
2. Observability (OpenTelemetry)	Distributed tracing across ASP.NET Core, HttpClient, SqlClient, and custom ActivitySource('SmartParking.Domain') spans. Exposes Prometheus metrics at /metrics and propagates X-Correlation-Id.
3. Rate Limiting	Partitioned by IP: auth-policy (Fixed Window: 10 req/min on /login & /register), write-policy (Token Bucket: 30 req/min on mutations), read-policy (Sliding Window: 120 req/min on queries). Returns HTTP 429 with Retry-After.
4. API Versioning	Supports both /api/SmartParking/* (default v1.0) and /api/v1/SmartParking/*, plus X-Api-Version header and ?api-version=1.0 query parameter.
5. Structured Logging	Serilog console & OpenTelemetry sinks enriched with CorrelationId, TraceId, and SpanId, paired with compile-time [LoggerMessage] source-generated logging methods.
6. Multi-Tier Caching	.NET 10 HybridCache (L1 in-memory + optional L2 Redis) and OutputCache with automatic tag eviction (clients, sites, buildings, floors, parkings-active) via CacheEvictionService on write operations.
7. Feature Management	Microsoft.FeatureManagement flags: EnableStrictSpotValidation, EnableDynamicSurgePricing (1.25x surge when floor occupancy >= 80%), and EnableDetailedAdminAnalytics.
8. Exception Handling	Chained IExceptionHandler pipeline (Validation -> 400, Conflict/Concurrency -> 409, NotFound -> 404, InvalidCredentials -> 401, Global -> 500) returning RFC 9457 ProblemDetails or ApiResponse envelopes.
9. Resilience	EF Core SQL Server EnableRetryOnFailure (5 retries) + Polly v8 'parking-checkin-pipeline' with exponential backoff + jitter on concurrency/deadlock errors, circuit breaker, and 5s timeout.
10. Swagger UI & Scalar	Interactive Swagger UI at /swagger (/swagger/v1/swagger.json) and Scalar API Reference at /scalar/v1 (/openapi/v1.json).

2.5 Complete Catalogue of All 33 Endpoints

Category	Method & Route (/api/SmartParking/...)	Payload / Params
Users & Auth (6)	GET /getAllUsersByClientId GET /getUserById POST /register POST /login PUT /updateUser DELETE /deleteUserById	?clientId={int} ?id={int} Body: UserMasterViewClient Body: UserLogin ?id={int}, Body: UserMasterNew ?id={int}
Clients (4)	GET /GetAllClients POST /AddClient POST /UpdateClient POST /DeleteClient	None Body: GetWClients Body: GetWClients ?id={int}

Category	Method & Route (/api/SmartParking/...)	Payload / Params
Sites & Buildings (7)	GET /GetAllSites GET /GetSitesByClientId GET /GetSiteById POST /AddClientSite POST /UpdateSite POST /DeleteSite GET /GetBuildingBySiteId	?clientid={int?} ?id={int} ?id={int} Body: GetVSiteMasterView Body: GetVSiteMasterView ?id={int} ?id={int}
Floors (6)	GET /GetAllFloor GET /GetFloorsByBuildingId GET /GetFloorsBySiteId POST /AddFloor POST /UpdateFloor POST /DeleteFloor	None ?id={int} ?id={int} Body: GetVFloors Body: GetVFloors ?id={int}
Parkings (10)	GET /GetAllParking GET /GetAllActiveParking GET /GetAllParkingByClientId GET /GetAllParkingBySite GET /GetAllParkingByBuilding GET /GetAllParkingByFloor POST /AddParking POST /MarkExit POST /DeleteParking POST /GetAdminAllParkings	None None ?id={int} ?id={int} ?id={int} ?id={int} Body: GetVParkings Body: ExitParking ?id={int} Body: AllParkingSearch

2.6 Key Engineering & Troubleshooting Lessons from Initial Bring-Up

- **Moving .NET Solutions Across Directories**: When moving the project to C:\Projects\SmartParking, all bin/ and obj/ folders were purged because NuGet caches absolute paths in obj/project.assets.json.
- **Development vs. Production Hosting Environment**: Without Properties/launchSettings.json, ASP.NET Core defaults to Production. Adding Properties/launchSettings.json sets ASPNETCORE_ENVIRONMENT=Development and opens /swagger automatically.
- **Globally Unique Endpoint Names in Minimal APIs**: Mapping the same endpoint methods to both /api/SmartParking and /api/v{version:apiVersion}/SmartParking requires passing a distinct namePrefix ('V1_') to .WithName(...) on the second route group to avoid Duplicate endpoint name exceptions.
- **MarkExit Endpoint Consolidation**: The upstream specification typo '/MarExit' was removed and merged into the single POST /api/SmartParking/MarkExit endpoint.

3. Step-by-Step Instructions to Run on Your Local Machine

Step 1: Verify Prerequisites (.NET 10 SDK & SQL Server LocalDB)

Open PowerShell and verify .NET 10 SDK and SQL Server LocalDB are available:

```
dotnet --list-sdks
sqllocaldb info mssqllocaldb
sqllocaldb start mssqllocaldb
```

Step 2: Navigate to the Solution Directory

```
cd C:\Projects\SmartParking
```

Step 3: Review Database & Feature Configuration

Inspect **C:\Projects\SmartParking\src\SmartParking.Api\appsettings.json**:
• **ConnectionStrings:SmartParkingDb** defaults to (localdb)\mssqllocaldb (database SmartParkingDb). Update this if you use a full SQL Server instance.
• **Database:AutoMigrateAndSeed** is set to true, which automatically creates the database tables and seeds sample data on startup (including default admin login admin@metrosmartparking.com / Admin@123).
• To execute the DDL script manually in SSMS, run: src\SmartParking.Infrastructure\Persistence\Scripts\001_InitialSchema.sql.

Step 4: Restore & Build the Solution

```
dotnet build "C:\Projects\SmartParking\SmartParking.slnx" -c Release
```

Confirm the build output shows: **Build succeeded. 0 Warning(s), 0 Error(s)**.

Step 5: Run the Application

```
dotnet run --project "C:\Projects\SmartParking\src\SmartParking.Api\SmartParking.Api.csproj"
```

Confirm the console output shows **Hosting environment: Development** and **Now listening on: http://localhost:5000**.

Step 6: Explore & Test the Endpoints

- **Swagger UI (Interactive Tester)**: http://localhost:5000/swagger
- **Swagger OpenAPI JSON**: http://localhost:5000/swagger/v1/swagger.json
- **Scalar API Reference**: http://localhost:5000/scalar/v1
- **Liveness Health Probe**: http://localhost:5000/health/live
- **Readiness Health Probe (SQL Server + EF Core)**: http://localhost:5000/health/ready
- **Prometheus Metrics Scraping Endpoint**: http://localhost:5000/metrics
- **IDE HTTP Requests Runner**: Open C:\Projects\SmartParking\src\SmartParking.Api\SmartParking.Api.http in Visual Studio / Rider / VS Code.

Day-1 Verification Checklist for New Engineers

[ ] Build C:\Projects\SmartParking\SmartParking.slnx with 0 Warning(s), 0 Error(s).

[ ] Start SmartParking.Api and verify /health/live and /health/ready return 'Healthy'.

[ ] Open http://localhost:5000/swagger and execute a complete parking lifecycle (AddClient -> AddClientSite -> AddFloor -> AddParking -> GetAllActiveParking -> MarkExit).

[ ] Test duplicate spot protection by calling AddParking twice for the same floorId and parkSpotNo while active, and verify HTTP 409 Conflict (ProblemDetails) is returned.

[ ] Inspect http://localhost:5000/metrics to confirm smartparking_parkings_checkin_count_total and smartparking_parkings_duration_minutes are recorded.

4. Complete Original Master Prompt (Verbatim)

Below is the complete, unedited architectural prompt used to design and build the Smart Parking Web API:

```
# System Role & Objective

Act as a **Principal .NET Architect and Staff Backend Engineer**. Your task is to design and
  implement a **complete, production-grade, high-performance Smart Parking Web API** using
  **.NET 10 (`net10.0`, C# 14)** and **Microsoft SQL Server** (EF Core 10 + Dapper/Compiled
  Queries for hot read paths).

The API must be **100% contract-compatible** with the Smart Parking API specification from
  `https://freeprojectapi.com/apis/smart-parking` (all 33 endpoints and 12 payload schemas
  listed below) while implementing enterprise-grade architecture, database design, performance
  optimizations, and cross-cutting production concerns:
1. **Health Checks & Custom Metrics**
2. **Full Observability (OpenTelemetry Tracing, Metrics, Logs)**
3. **Rate Limiting**
4. **API Versioning**
5. **Structured High-Performance Logging**
6. **Multi-Tier Caching (`HybridCache` + `OutputCache` with Tag Invalidation)**
7. **Feature Management (`Microsoft.FeatureManagement`)**
8. **Centralized Exception Handling (`IExceptionHandler` + RFC 9457 `ProblemDetails` +
  `ApiResponse` envelope support)**
9. **Resilience (`Microsoft.Extensions.Resilience` / Polly v8 + EF Core Execution Strategies)**

---

## 1. Architecture & Performance Guidelines (.NET 10 Best Practices)

- **Clean / Vertical Slice Architecture**:
  - `SmartParking.Domain`: Entities, value objects, domain exceptions, enums, auditing
  interfaces (`IAuditableEntity`, `ISoftDeletable`).
  - `SmartParking.Application`: CQRS/Vertical Slice handlers, DTOs/contracts matching the exact
  JSON property names, `FluentValidation` validators, feature flag definitions, caching
  abstractions, custom metric counters/histograms.
  - `SmartParking.Infrastructure`: EF Core 10 `SmartParkingDbContext`,
  `IEntityTypeConfiguration<T>` classes, SQL Server migrations, compiled queries, `HybridCache`
  & Redis configuration, resilience pipelines, OpenTelemetry & Serilog setup.
  - `SmartParking.Api`: Minimal APIs (with `IEndpointRouteBuilder` endpoint groups) or high-
  performance Controllers, middleware pipeline, `IExceptionHandler` implementations, Rate
  Limiting, API Versioning, Health Check endpoints, and `.NET 10 OpenAPI` + `Scalar` API
  documentation.
- **High-Performance Data Access**:
  - Default `QueryTrackingBehavior.NoTracking` on read queries.
  - Use **EF Core 10 Compiled Queries** (`EF.CompileAsyncQuery`) or **Dapper** for high-
  frequency endpoints (`GetAllActiveParking`, `GetAllParkingBySite`, `GetAllFloor`).
  - Use projection (`Select(...)`) directly to response DTOs in SQL—never load full entity
  graphs into memory for read endpoints.
  - Use `System.Text.Json` source generation (`JsonSerializerContext`) for low-allocation
  serialization.
  - Enforce `CancellationToken` propagation across every async method signature from HTTP
  request down to SQL Server execution.

---

## 2. SQL Server Database Schema & Indexing Strategy

Design normalized, high-performance SQL Server tables under schema `dbo` with proper foreign
  keys, constraints, `rowversion` concurrency tokens, and indexes that back all 33 API
  endpoints:

### Tables
1. **`Clients`**
   - `ClientId` (`INT IDENTITY(1,1) PRIMARY KEY`)
   - `ClientName` (`NVARCHAR(150) NOT NULL`)
   - `BusinessName` (`NVARCHAR(200) NULL`)
   - `ContactPerson` (`NVARCHAR(150) NULL`)
   - `ContactNo` (`VARCHAR(25) NULL`)
   - `AltContactNo` (`VARCHAR(25) NULL`)
   - `Email` (`NVARCHAR(256) NULL`)
   - `CreatedDate` (`DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME()`)
   - `Logo` (`NVARCHAR(500) NULL`)
```

```
   - `IsDeleted` (`BIT NOT NULL DEFAULT 0`)

2. **`Users`**
   - `UserId` (`INT IDENTITY(1,1) PRIMARY KEY`)
   - `ClientId` (`INT NULL`, FK -> `Clients.ClientId` ON DELETE SET NULL)
   - `EmailId` (`NVARCHAR(256) NOT NULL`)
   - `PasswordHash` (`NVARCHAR(512) NOT NULL`) — *Use `IPasswordHasher<User>` / PBKDF2 or
  Argon2id; never store plain text*
   - `FullName` (`NVARCHAR(150) NOT NULL`)
   - `MobileNo` (`VARCHAR(25) NOT NULL`)
   - `ProjectName` (`NVARCHAR(100) NULL DEFAULT 'SmartParking'`)
   - `ExtraId` (`INT NULL`)
   - `CreatedDate` (`DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME()`)

3. **`Sites`**
   - `SiteId` (`INT IDENTITY(1,1) PRIMARY KEY`)
   - `ClientId` (`INT NOT NULL`, FK -> `Clients.ClientId` ON DELETE CASCADE)
   - `SiteName` (`NVARCHAR(200) NOT NULL`)
   - `SiteCity` (`NVARCHAR(100) NULL`)
   - `SiteAddress` (`NVARCHAR(500) NULL`)
   - `SitePinCode` (`VARCHAR(20) NULL`)
   - `TotalBuildings` (`INT NOT NULL DEFAULT 0`)
   - `CreatedDate` (`DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME()`)

4. **`Buildings`**
   - `BuildingId` (`INT IDENTITY(1,1) PRIMARY KEY`)
   - `SiteId` (`INT NOT NULL`, FK -> `Sites.SiteId` ON DELETE CASCADE)
   - `BuildingName` (`NVARCHAR(150) NOT NULL`)
   - `BuildingManagerName` (`NVARCHAR(150) NULL`)
   - `ContactNo` (`VARCHAR(25) NULL`)

5. **`Floors`**
   - `FloorId` (`INT IDENTITY(1,1) PRIMARY KEY`)
   - `BuildingId` (`INT NOT NULL`, FK -> `Buildings.BuildingId` ON DELETE CASCADE)
   - `FloorNo` (`NVARCHAR(50) NOT NULL`)
   - `IsOperational` (`BIT NOT NULL DEFAULT 1`)
   - `TotalParkingSpots` (`INT NOT NULL CHECK (TotalParkingSpots >= 0)`)
   - `RowVersion` (`ROWVERSION NOT NULL`)

6. **`Parkings`**
   - `ParkId` (`INT IDENTITY(1,1) PRIMARY KEY`)
   - `FloorId` (`INT NOT NULL`, FK -> `Floors.FloorId`)
   - `CustName` (`NVARCHAR(150) NULL`)
   - `CustMobileNo` (`VARCHAR(25) NULL`)
   - `VehicleNo` (`NVARCHAR(50) NOT NULL`)
   - `ParkDate` (`DATETIME2(3) NOT NULL`)
   - `ParkSpotNo` (`INT NOT NULL`)
   - `InTime` (`DATETIME2(3) NOT NULL`)
   - `OutTime` (`DATETIME2(3) NULL`)
   - `Amount` (`DECIMAL(18,2) NOT NULL DEFAULT 0`)
   - `ExtraCharge` (`DECIMAL(18,2) NOT NULL DEFAULT 0`)
   - `ParkingNo` (`NVARCHAR(50) NULL`)
   - `RowVersion` (`ROWVERSION NOT NULL`)

### Required SQL Server Indexes
- **`UX_Users_EmailId`**: Unique non-clustered index on `Users(EmailId)`.
- **`IX_Users_ClientId`**: Non-clustered index on `Users(ClientId) INCLUDE (EmailId, FullName,
  MobileNo)`.
- **`IX_Sites_ClientId`**: Non-clustered index on `Sites(ClientId) INCLUDE (SiteName, SiteCity,
  TotalBuildings)`.
- **`IX_Buildings_SiteId`**: Non-clustered index on `Buildings(SiteId) INCLUDE (BuildingName,
  BuildingManagerName, ContactNo)`.
- **`IX_Floors_BuildingId`**: Non-clustered index on `Floors(BuildingId) INCLUDE (FloorNo,
  IsOperational, TotalParkingSpots)`.
- **`UX_Parkings_ActiveSpot` (Critical Filtered Unique Index)**:
  ```sql
  CREATE UNIQUE NONCLUSTERED INDEX UX_Parkings_ActiveSpot
  ON dbo.Parkings (FloorId, ParkSpotNo)
  WHERE OutTime IS NULL;
  ```
  *Guarantees at the database engine level that two vehicles cannot simultaneously occupy the
  same spot on the same floor, and accelerates `GetAllActiveParking`.*
- **`IX_Parkings_Search` (Covering Index for Admin Search & Hierarchy Queries)**:
```

  ```sql
  CREATE NONCLUSTERED INDEX IX_Parkings_Search
  ON dbo.Parkings (FloorId, ParkDate DESC, VehicleNo)
  INCLUDE (CustName, CustMobileNo, ParkSpotNo, InTime, OutTime, Amount, ExtraCharge,
  ParkingNo);
  ```

---

3. Exact API Contracts & Schemas (`freeprojectapi.com/apis/smart-parking`)

Preserve exact camelCase JSON property names for frontend compatibility:

Standard Envelope (`ApiResponse`)
```json
{
  "message": "string",
  "result": true,
  "data": {}
}
```

Request / Response DTO Schemas
- **`UserLogin`**: `{ "emailId": "string", "password": "string" }`
- **`UserMasterView`**: `{ "userId": 0, "emailId": "string", "password": "string", "fullName":
  "string", "mobileNo": "string" }`
- **`UserMasterViewClient`**: `{ "userId": 0, "emailId": "string", "password": "string",
  "fullName": "string", "mobileNo": "string", "clientId": 0 }`
- **`UserMasterNew`**: `{ "userId": 0, "emailId": "string", "password": "string",
  "createdDate": "2026-01-01T00:00:00Z", "projectName": "string", "fullName": "string",
  "mobileNo": "string", "extraId": 0 }` *(Note: Redact `password` in responses or return
  `"***"` while keeping property present for schema compatibility)*.
- **`GetWClients`**: `{ "clientId": 0, "clientName": "string", "businessName": "string",
  "contactPerson": "string", "contactNo": "string", "altContactNo": "string", "email":
  "string", "createdDate": "2026-01-01T00:00:00Z", "logo": "string" }`
- **`GetVBuildings`**: `{ "buildingId": 0, "siteId": 0, "buildingName": "string",
  "buildingManagerName": "string", "contactNo": "string" }`
- **`GetVSiteMasterView`**:
  ```json
  {
    "siteId": 0,
    "clientId": 0,
    "siteName": "string",
    "siteCity": "string",
    "siteAddress": "string",
    "sitePinCode": "string",
    "totalBuildings": 0,
    "createdDate": "2026-01-01T00:00:00Z",
    "getVBuildingss": [ /* Array of GetVBuildings (note the exact double-'s' property name from
  spec) */ ]
  }
  ```
- **`GetVFloors`**: `{ "floorId": 0, "buildingId": 0, "floorNo": "string", "isOperational":
  true, "totalParkingSpots": 0 }`
  - *Read projection (`FloorListItemDto`)* also includes joined `"buildingName": "string"` and
  `"siteName": "string"`.
- **`GetVParkings`**:
  ```json
  {
    "parkId": 0,
    "floorId": 0,
    "custName": "string",
    "custMobileNo": "string",
    "vehicleNo": "string",
    "parkDate": "2026-01-01T00:00:00Z",
    "parkSpotNo": 0,
    "inTime": "2026-01-01T00:00:00Z",
    "outTime": "2026-01-01T00:00:00Z",
    "amount": 0.0,
    "extraCharge": 0.0,
    "parkingNo": "string"
  }
  ```

  - *Read projection (`ParkingListItemDto`)* also includes joined `"floorNo": "string"`, `"buildingName": "string"`, `"siteName": "string"`, and `"clientName": "string"`.
- **`ExitParking`**: `{ "parkId": 0, "outTime": "2026-01-01T00:00:00Z", "extraCharge": 0.0 }`
- **`AllParkingSearch`**: `{ "clientId": 0, "siteId": 0, "buildingId": 0, "floorId": 0, "fromDate": "2026-01-01T00:00:00Z", "toDate": "2026-01-01T00:00:00Z", "vehicleNo": "string" }` *(Treat `0` or `null` filter values as optional/wildcard in the dynamic query)*.

---

4. All 33 Endpoints to Implement

A. User & Auth Endpoints
1. `GET /api/SmartParking/getAllUsersByClientId?clientId={int}` -> Returns `UserMasterView[]`
2. `GET /api/SmartParking/getUserById?id={int}` -> Returns `UserMasterNew`
3. `POST /api/SmartParking/register` (Body: `UserMasterViewClient`) -> Hashes password, creates user linked to `clientId`, returns `UserMasterNew`
4. `POST /api/SmartParking/login` (Body: `UserLogin`) -> Verifies credentials, returns `UserMasterNew` (and optionally issues JWT token in header/response while preserving `UserMasterNew` fields)
5. `PUT /api/SmartParking/updateUser?id={int}` (Body: `UserMasterNew`) -> Updates user record
6. `DELETE /api/SmartParking/deleteUserById?id={int}` -> Deletes user record

B. Client Endpoints
7. `GET /api/SmartParking/GetAllClients` -> Returns `ApiResponse` with `data = List<GetWClients>`
8. `POST /api/SmartParking/AddClient` (Body: `GetWClients`) -> Creates client, invalidates client cache tag, returns `ApiResponse`
9. `POST /api/SmartParking/UpdateClient` (Body: `GetWClients`) -> Updates client, invalidates client cache tag, returns `ApiResponse`
10. `POST /api/SmartParking/DeleteClient?id={int}` -> Soft-deletes or deletes client, invalidates cache, returns `ApiResponse`

C. Site & Building Endpoints
11. `GET /api/SmartParking/GetAllSites?clientid={int?}` -> Returns `ApiResponse` with sites (filtered by `clientid` if `> 0`, including `clientName`)
12. `GET /api/SmartParking/GetSitesByClientId?id={int}` -> Returns `ApiResponse` with sites belonging to client `id`
13. `GET /api/SmartParking/GetSiteById?id={int}` -> Returns `ApiResponse` with `GetVSiteMasterView` including nested `getVBuildingss`
14. `POST /api/SmartParking/AddClientSite` (Body: `GetVSiteMasterView`) -> Transactional insert of `Site` + nested `getVBuildingss` into `Buildings`, returns `ApiResponse`
15. `POST /api/SmartParking/UpdateSite` (Body: `GetVSiteMasterView`) -> Transactional update of `Site` + upsert/sync of nested `getVBuildingss`, returns `ApiResponse`
16. `POST /api/SmartParking/DeleteSite?id={int}` -> Deletes site, invalidates cache, returns `ApiResponse`
17. `GET /api/SmartParking/GetBuildingBySiteId?id={int}` -> Returns `ApiResponse` with `List<GetVBuildings>` for the given `siteId`

D. Floor Endpoints
18. `GET /api/SmartParking/GetAllFloor` -> Returns `ApiResponse` with all floors including `buildingName` and `siteName`
19. `GET /api/SmartParking/GetFloorsByBuildingId?id={int}` -> Returns `ApiResponse` with floors for `buildingId`
20. `GET /api/SmartParking/GetFloorsBySiteId?id={int}` -> Returns `ApiResponse` with floors across all buildings in `siteId`
21. `POST /api/SmartParking/AddFloor` (Body: `GetVFloors`) -> Creates floor, invalidates floor cache tag, returns `ApiResponse`
22. `POST /api/SmartParking/UpdateFloor` (Body: `GetVFloors`) -> Updates floor, invalidates floor cache tag, returns `ApiResponse`
23. `POST /api/SmartParking/DeleteFloor?id={int}` -> Deletes floor, invalidates floor cache tag, returns `ApiResponse`

E. Parking Operations & Admin Search Endpoints
24. `GET /api/SmartParking/GetAllParking` -> Returns `ApiResponse` with all parking records joined with `floorNo`, `buildingName`, `siteName`, `clientName`
25. `GET /api/SmartParking/GetAllActiveParking` -> Returns `ApiResponse` with active parkings (`OutTime IS NULL`)
26. `GET /api/SmartParking/GetAllParkingByClientId?id={int}` -> Returns `ApiResponse` filtered by `ClientId`
27. `GET /api/SmartParking/GetAllParkingBySite?id={int}` -> Returns `ApiResponse` filtered by `SiteId`
28. `GET /api/SmartParking/GetAllParkingByBuilding?id={int}` -> Returns `ApiResponse` filtered by `BuildingId`

29. `GET /api/SmartParking/GetAllParkingByFloor?id={int}` -> Returns `ApiResponse` filtered by `FloorId`
30. `POST /api/SmartParking/AddParking` (Body: `GetVParkings`) -> Checks that floor is operational (`IsOperational == true`), spot `ParkSpotNo` is within `[1..TotalParkingSpots]` and not currently occupied (`OutTime IS NULL`), calculates dynamic/surge pricing if feature flag is enabled, saves ticket, increments custom metrics, evicts active parking cache, and returns `ApiResponse`
31. `POST /api/SmartParking/MarkExit` (Body: `ExitParking`) -> Finds active parking by `parkId`, sets `OutTime` and `ExtraCharge`, records parking duration histogram metric, evicts active parking cache, and returns `ApiResponse`
32. `POST /api/SmartParking/DeleteParking?id={int}` -> Deletes parking record, evicts cache, returns `ApiResponse`
33. `POST /api/SmartParking/GetAdminAllParkings` (Body: `AllParkingSearch`) -> High-performance dynamic predicate query filtering by optional `clientId`, `siteId`, `buildingId`, `floorId`, `fromDate`, `toDate`, and `vehicleNo` prefix/contains, returning `ApiResponse`

---

5. Mandatory Enterprise Cross-Cutting Requirements

1. Health Checks & Metrics
- Configure `AddHealthChecks()` with:
  - **Liveness (`/health/live`)**: Self check (`tags: ["live"]`), zero external dependencies.
  - **Readiness (`/health/ready`)**: SQL Server (`AddSqlServer` / `AddDbContextCheck<SmartParkingDbContext>`) and Redis/HybridCache health check (`tags: ["ready"]`), returning detailed JSON health report via custom `ResponseWriter`.
- Create a singleton `SmartParkingMetrics` service using `IMeterFactory` (`Meter("SmartParking.Api", "1.0.0")`):
  - `Counter<long>` `smartparking.parkings.checkin.count` (tagged by `site_id`, `building_id`, `floor_id`)
  - `Counter<long>` `smartparking.parkings.checkout.count`
  - `UpDownCounter<long>` `smartparking.parkings.active`
  - `Histogram<double>` `smartparking.parkings.duration.minutes`
  - `Counter<decimal>` `smartparking.revenue.collected`

2. Observability (OpenTelemetry)
- Configure `AddOpenTelemetry()`:
  - **Tracing**: `AddAspNetCoreInstrumentation()`, `AddHttpClientInstrumentation()`, `AddSqlClientInstrumentation(o => o.SetDbStatementForText = true)`, plus custom `ActivitySource("SmartParking.Domain")` spans around `AddParking` and `MarExit`.
  - **Metrics**: `AddAspNetCoreInstrumentation()`, `AddRuntimeInstrumentation()`, `AddMeter("SmartParking.Api")`, Prometheus scraping endpoint (`/metrics`) and OTLP exporter (`AddOtlpExporter()`).
  - **Correlation**: Middleware that reads/generates `X-Correlation-Id`, adds it to `Activity.Current`, response headers, and `LogContext`.

3. Rate Limiting
- Use `Microsoft.AspNetCore.RateLimiting`:
  - **`auth-policy`**: `FixedWindowLimiter` (10 requests / minute partitioned by client IP) applied to `/login` and `/register`.
  - **`write-policy`**: `TokenBucketLimiter` (30 requests / minute, burst of 10, partitioned by user/IP) applied to all `POST`, `PUT`, `DELETE` mutation endpoints.
  - **`read-policy`**: `SlidingWindowLimiter` (120 requests / minute partitioned by IP) applied to `GET` endpoints and `GetAdminAllParkings`.
  - Custom `OnRejected` handler returning HTTP `429 Too Many Requests` with `Retry-After` header and standardized JSON payload.

4. API Versioning
- Configure `Asp.Versioning.Http` (`AddApiVersioning` + `AddApiExplorer`):
  - Default version: `1.0` (`AssumeDefaultVersionWhenUnspecified = true`).
  - Support **both** unversioned legacy routes (`/api/SmartParking/...`) AND versioned routes (`/api/v{version:apiVersion}/SmartParking/...`), plus `HeaderApiVersionReader("X-Api-Version")` and `QueryStringApiVersionReader("api-version")`.
  - Report supported versions (`ReportApiVersions = true`).

5. Proper Structured Logging
- Configure **Serilog** (`UseSerilog`) with Console (JSON in Production, themed in Development) and OpenTelemetry/Seq sinks.
- Enrich with `FromLogContext`, `WithMachineName`, `WithEnvironmentName`, `WithSpan` (TraceId/SpanId).
- Implement compile-time `[LoggerMessage]` source-generated extension methods for high-throughput logging without boxing or string interpolation overhead.
- Never log sensitive fields (`password` in `UserLogin`, `UserMasterNew`, `UserMasterView`,

`UserMasterViewClient`).

6. Caching (`HybridCache` + `OutputCache`)
- Register .NET **`HybridCache`** (`AddHybridCache()`) backed by `IMemoryCache` (L1) and Redis/Distributed SQL Cache (L2) with stampede protection.
- Cache reference/hierarchy queries with tags:
  - `GetAllClients` -> tag: `"clients"` (TTL: 10 min L1 / 30 min L2)
  - `GetAllSites`, `GetSitesByClientId`, `GetSiteById`, `GetBuildingBySiteId` -> tags: `["sites", $"site:{id}", $"client-sites:{clientId}"]` (TTL: 5 min)
  - `GetAllFloor`, `GetFloorsByBuildingId`, `GetFloorsBySiteId` -> tags: `["floors", $"building-floors:{buildingId}", $"site-floors:{siteId}"]` (TTL: 5 min)
  - `GetAllActiveParking` -> tag: `"parkings-active"` (short TTL: 15 sec L1)
- Call `await hybridCache.RemoveByTagAsync(...)` and `await outputCacheStore.EvictByTagAsync(...)` automatically inside write handlers (`AddClient`, `UpdateClient`, `DeleteClient`, `AddClientSite`, `UpdateSite`, `DeleteSite`, `AddFloor`, `UpdateFloor`, `DeleteFloor`, `AddParking`, `MarExit`, `DeleteParking`).

7. Feature Management
- Use `Microsoft.FeatureManagement.AspNetCore` (`AddFeatureManagement()`):
  - `EnableDynamicSurgePricing`: Automatically applies a configurable surge multiplier to `Amount` in `AddParking` when floor occupancy exceeds 80%.
  - `EnableStrictSpotValidation`: Enforces strict spot bounds (`1 <= ParkSpotNo <= TotalParkingSpots`) and operational floor checks before check-in.
  - `EnableDetailedAdminAnalytics`: Enriches `GetAdminAllParkings` with summary metadata or enables advanced search filters.
- Demonstrate usage via `IFeatureManagerSnapshot` in application handlers and endpoint filters.

8. Exception Handling Management
- Register custom `IExceptionHandler` implementations chained in order:
  1. `ValidationExceptionHandler` -> Maps `FluentValidation.ValidationException` to HTTP `400 Bad Request`.
  2. `ParkingConflictExceptionHandler` -> Maps spot-already-occupied / `DbUpdateConcurrencyException` / unique constraint violation (`2601`/`2627`) to HTTP `409 Conflict`.
  3. `EntityNotFoundExceptionHandler` -> Maps not-found domain exceptions to HTTP `404 Not Found`.
  4. `GlobalExceptionHandler` -> Catch-all logging full exception with `TraceId` and returning HTTP `500 Internal Server Error` without leaking stack traces in Production.
- Support both standard **RFC 9457 `ProblemDetails`** (`application/problem+json`) and an `ApiResponse` compatibility mode (`{ "result": false, "message": "...", "data": null }`) when requested via header or query.

9. Resilience
- **Database Resilience**: Configure EF Core `UseSqlServer(connectionString, sql => sql.EnableRetryOnFailure(maxRetryCount: 5, maxRetryDelay: TimeSpan.FromSeconds(10), errorNumbersToAdd: null))`.
- **Application Resilience Pipelines**: Register `AddResiliencePipeline("parking-checkin-pipeline", ...)` using `Microsoft.Extensions.Resilience` (Polly v8) with:
  - Optimistic concurrency retry (`DbUpdateConcurrencyException`, up to 3 attempts with 50ms exponential backoff + jitter).
  - Timeout strategy (5 seconds per operation).
  - Circuit breaker for downstream dependencies.

---

6. Expected Deliverables & Output Structure

Provide a complete, buildable, well-structured implementation containing:
1. **Solution & `.csproj` files** targeting `net10.0` with exact NuGet package versions.
2. **SQL Server DDL Script & EF Core 10 `DbContext` + Entity Configurations** (including the filtered unique index `UX_Parkings_ActiveSpot` and seed data).
3. **Complete DTOs, Validators (`FluentValidation`), and Compiled Queries**.
4. **All 33 Endpoints** organized cleanly into modular endpoint groups (`UserEndpoints`, `ClientEndpoints`, `SiteEndpoints`, `FloorEndpoints`, `ParkingEndpoints`).
5. **Cross-Cutting Infrastructure Code**:
   - `SmartParkingMetrics.cs`
   - `ExceptionHandlers.cs`
   - `CachingService` / `HybridCache` tag helpers
   - `FeatureFlags.cs`
   - `Program.cs` wiring OpenTelemetry, Serilog, Rate Limiting, API Versioning, `HybridCache`, `OutputCache`, Feature Management, Resilience, Health Checks, and OpenAPI/Scalar.
6. **`appsettings.json`** showing production-ready configuration for SQL Server, Serilog, OpenTelemetry, RateLimiting, HybridCache, and `FeatureManagement`.